

Week 8 | 课时 3 | 混合检索: pgvector + PostgreSQL FTS + RRF + Metadata Filter

本课导航

只向量检索不够，企业知识需要语义、关键词、权限和版本一起约束	1
这节课解决什么问题	1
参考学习时间	1
先看一张总图	2
1. vector search 和 FTS 各自解决什么	2
2. Metadata filter 必须在生成前生效	2
3. RRF 的作用	2
4. Debug score 是排查入口	3
5. 检索结果必须携带 evidence metadata	3
本课最重要判断	3
自检清单	3

只向量检索不够，企业知识需要语义、关键词、权限和版本一起约束

这一讲把检索主链路讲清楚: vector search、FTS search、RRF fusion、metadata filters 和 debug scores。

这节课解决什么问题

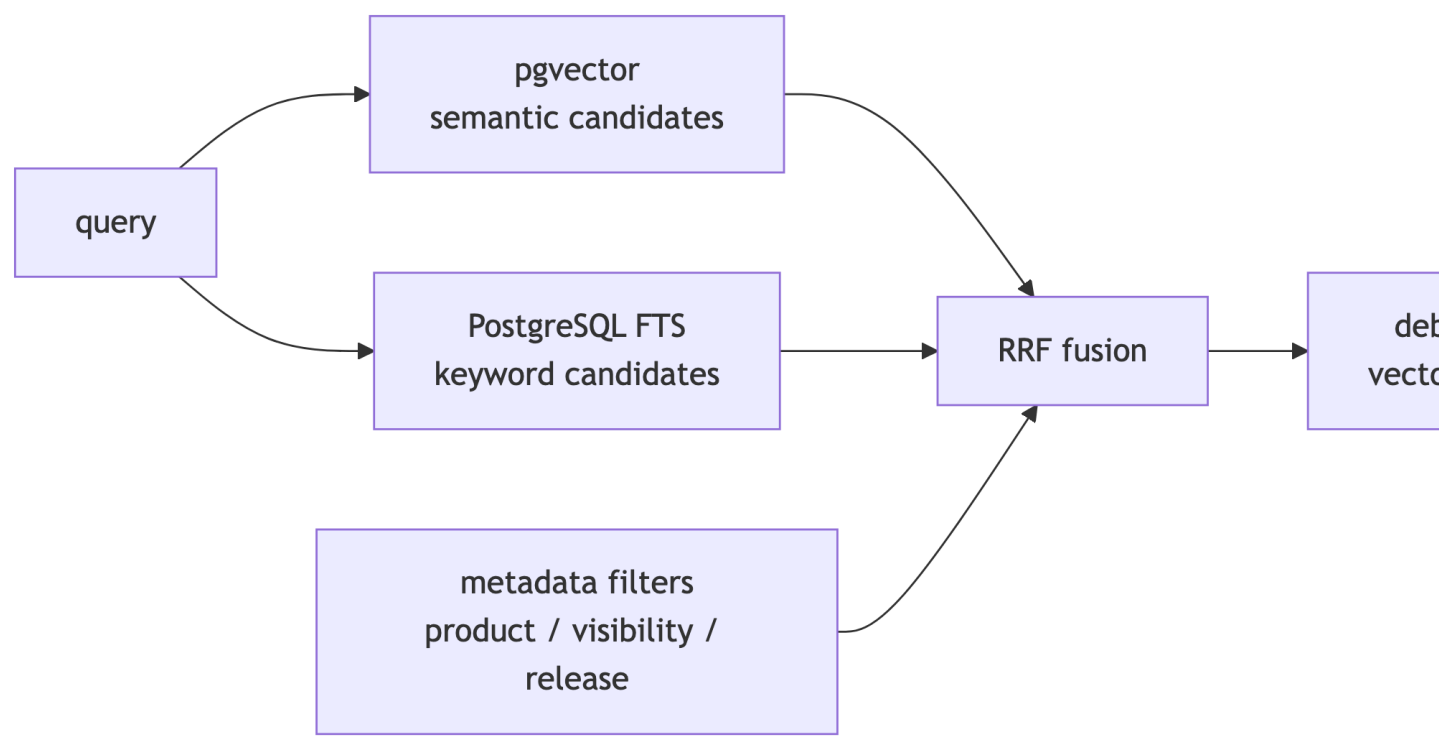
向量检索擅长语义相似，但在企业支持场景里，很多问题靠关键词和过滤条件决定生死：

- 产品型号
- 固件版本
- 错误码
- 可见范围
- 权限层级
- 数据和索引版本

参考学习时间

45—55 分钟

先看一张总图



1. vector search 和 FTS 各自解决什么

检索方式	擅长	不擅长
pgvector	语义近似、同义表达、长问题	精确型号、版本号、权限过滤
PostgreSQL FTS	关键词、错误码、短语、术语	语义改写、长上下文
RRF	合并多路排序，降低单一路径偏差	不替代 evidence gate

2. Metadata filter 必须在生成前生效

Week8 至少要支持：

- product_line
- visibility_scope
- entitlement_tier
- status
- quality_status
- data_release_id
- index_release_id

! 核心判断

权限、可见范围和 release 过滤必须发生在 retrieval 阶段。生成之后再过滤，已经太晚。

3. RRF 的作用

RRF 的基本公式：

```
score += 1 / (k + rank)
```

默认 `k=60`。它不要求 `vector score` 和 `FTS score` 在同一个数值尺度上，而是用排序位置融合候选结果。

4. Debug score 是排查入口

一个检索结果至少要能解释：

字段	用途
<code>vector_score</code>	语义相似度线索
<code>fts_score</code>	关键词命中线索
<code>rrf_score</code>	融合排序线索
<code>rerank_score</code>	二阶段精排线索
<code>final_score</code>	生成门禁使用的最终分

5. 检索结果必须携带 evidence metadata

检索结果不能只返回 `content`。

还必须返回：

- `evidence_id`
- `chunk_id / section_id`
- `doc_id`
- `source_id`
- `page_no`
- `section_path`
- `bbox`
- `source_url`
- `doc_version`

否则 Lesson 4 和 Lesson 5 无法构造可信 citation。

本课最重要判断

混合检索不是“向量 + 关键词越多越好”，而是让每条候选证据都能解释来源、过滤、分数和版本。

自检清单

- 我能解释为什么只向量检索不够。
- 我能说清 RRF 为什么适合融合不同检索分数。
- 我知道 metadata filter 必须在生成前执行。
- 我能读懂 retrieval debug 里的 score breakdown。